Fundamentos de Sistemas Operacionais

Sincronização

Prof. Msc. Lucas Reis

lucas.oliveira@puc-campinas.edu.br

Campinas, 2s/2026



Nesta aula…

- Competição de recursos para múltiplas tarefas (threads ou processos) acessando a mesma memória
  - Como solucionar
- Conceitos chave
  - **Condição de corrida**
  - **Memória compartilhada**
  - **Solução de Peterson**
  - **Variáveis atômicas, semáforos e mutexes**





Background

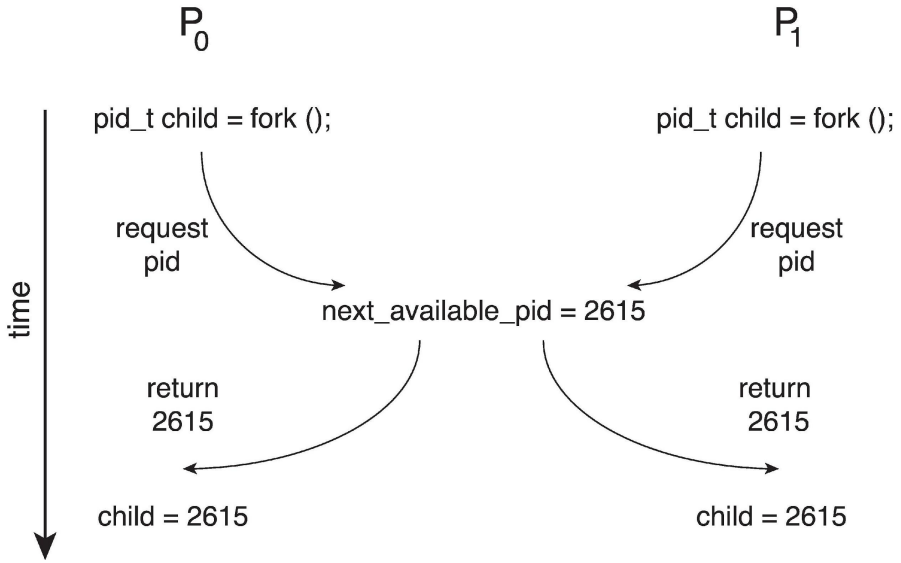
- Processos podem ser executados concorrentemente
  - Podem ser interrompidos (preempção) a qualquer instante
- Acesso concorrente a dados compartilhados pode gerar conflitos
- Manter consistência de dados requer mecanismos para garantir a ordem de execução de processos cooperantes



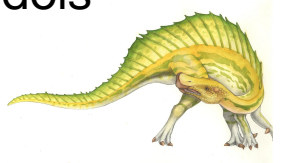


Condição de Corrida

- Processos P_0 e P_1 estão criando processos filhos com a chamada de sistema `fork()`
- Condição de corrida na variável do kernel `next_available_pid` que representa o próximo identificador de processo (pid) disponível



- Sem um mecanismo para prevenir P_0 e P_1 de acessar a variável `next_available_pid`, o mesmo pid poderia ser designado a dois processos diferentes!





Problema da Seção Crítica

- Considere um sistema de ***n*** processos $\{p_0, p_1, \ldots p_{n-1}\}$
- Cada processo tem um segmento de código denominado **seção crítica**
  - Processos podem estar alterando variáveis compartilhadas, atualizando structs, escrevendo em arquivos, etc.
  - Quando um processo está em sua seção crítica, nenhum outro deve estar na seção crítica
- O ***problema da seção crítica*** é a proposta de um protocolo para resolver isso
- Cada processo deve pedir permissão para entrar na seção crítica, para então executá-la, e por fim sair da seção crítica





Seção Crítica

- Estrutura padrão para processo P_i

```
while (true) {
        entry section
            critical section
        exit section
            remainder section
}
```





Problema da Seção Crítica

Requisitos para uma solução do problema da Seção Crítica

1. **Exclusão Mútua** - Se o processo P_i está executando sua seção crítica, nenhum outro processo pode acessar sua seção crítica
2. **Progresso** - Se nenhum processo está executando sua seção crítica e existem processos que gostariam de entrar em sua seção crítica, a seleção de qual processo pode entrar não pode ser adiada indefinidamente
3. **Espera Limitada** - Deve existir um limite para o número de vezes que outros processos podem entrar em suas seções críticas após um processo P_i requisitar o acesso a sua seção crítica.
   - Assumimos que todos os processos executam com velocidade maior que zero
   - Sem assumir nada sobre a **velocidade relativa** dos ***n*** processos





Solução Ingênua

- Problema de dois processos (P_i e P_j)
- Assume que **`load`** e **`store`** são instruções atômicas (não podem ser interrompidas)
- Dois processos compartilham uma variável:
  - **`int turn;`**
- A variável **`turn`** indica o turno de quem pode entrar na seção crítica
- Inicialmente, **`turn`** é setado para *i*





Algoritmo para Processo P_i

```
while (true){

    while (turn = = j);

    /* critical section */

    turn = j;

    /* remainder section */

}
```





Corretude da Solução Ingênua

- Exclusão mútua é preservada

  P_i entra na seção crítica apenas se:

  `turn = i`

  e `turn` não pode ser `i` e `j` ao mesmo tempo
- E o requisito de progresso?
  - Sim!
- E o requisito de espera limitada?
  - **Não!**





Solução de Peterson

- Solução de dois processos (P_i e P_j)
- Dois processos compartilham as seguintes variáveis:
  - `int turn;`
  - `boolean flag[2]`
- A variável **`turn`** indica o turno de quem pode entrar na seção crítica
- O vetor **`flag`** é usado para indicar se um processo está pronto para entrar na seção crítica.
- **`flag[i] = true`** implica que o processo **P_i** está pronto!





Algoritmo para Processo P_i

```
while (true){

    flag[i] = true;
    turn = j;
    while (flag[j] && turn = = j)
        ;

        /* critical section */

    flag[i] = false;

    /* remainder section */

}
```





Corretude da Solução de Peterson

- Prova que os três requisitos são cumpritos:
  1. Exclusão mútua é preservada

     P_i entra na SC apenas se:

     ou **`flag[j] = false`** ou **`turn = i`**
  2. Requisito de progresso é satisfeito
  3. Requisito de espera limitada é satisfeito

- Assume que **`load`** e **`store`** são instruções atômicas (não podem ser interrompidas)
  - **Não é verdade na prática**





Resumindo condições de corrida

- Para programas **multithread:**
  - Variáveis locais: cada thread tem sua versão
  - Variáveis globais: compartilhadas, pode haver condição de corrida
- Quando condições de corrida ocorrem
  - Se uma ou mais thread escreve em uma região de memória compartilhada ou
  - Se uma thread escreve e ao menos uma lê uma região de memória compartilhada
- Exemplos:
  - **Thread 1 escreve, Thread 2 lê** **OCORRE**
  - **Thread 1 escreve, Threads 2, 3, 4, … lêem** **OCORRE**
  - **Threads 1 e 2 lêem** **NÃO OCORRE**
  - **Threads 1 e 2 escrevem** **OCORRE**





Exemplo em C

```c
int x; // global/compartilhada

void * thread_run(void *args) {
  int i; // não compartilhada (OK!)
  i++; // não gera condição de corrida
  x++; // gera condição de corrida!
}
```





Hardware de Sincronização

- Sistemas oferecem instruções de hardware que permitem o acesso *atômico* à memória (**load** e **store**)
  - Instrução **Test-and-Set**
  - Instrução **Compare-and-Swap**





Variáveis Atômicas

- **Variável atômica** provê atualizações *atômicas* (ininterruptas) em tipos básicos de dados como inteiros e booleanos.
- Exemplo:
  - Seja **`sequence`** uma variável atômica
  - Seja **`increment()`** uma operação na variável atômica **`sequence`**
  - O comando:

```
increment(&sequence);
```

garante que **`sequence`** é incrementada sem interrupções





Locks Mutex

- Soluções anteriores são complicadas e pouco acessíveis para programadores
- Designers de SO provêem ferramentas de software para solucionar o problema da seção crítica
- A mais simples é o mutex lock
  - Variável booleana indicando se um lock está disponível
- Protegendo a seção crítica:
  - Primeiro **`acquire()`** um lock
  - Depois **`release()`** o lock
- Chamadas para **`acquire()`** e **`release()`** devem ser **atômicas**
  - Geralmente implementadas usando instruções atômicas de hardware como *compare-and-swap*.
- Essa solução implica em **busy waiting**
  - O lock é chamado de **spinlock**





Solution para seção crítica usando Locks Mutex

```
while (true) {

    acquire lock

        critical section

    release lock

remainder section
}
```





Semáforos

- Forma mais sofisticada de mutex para oferecer sincronização.
- Semáforo ***S*** – variável inteira
- Só pode ser acessado por duas operações atômicos
  - **`wait()`** e **`signal()`**
- Definição da operação **`wait()`**

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

- Definição da operação **`signal()`**

```
signal(S) {
    S++;
}
```





Semáforo

- **Semáforo de contagem** – inteiro que varia pelo número possível de recursos
- **Semáforo binário** – inteiro que varia entre 0 e 1
  - Como um **mutex**
- Com semáforos, podemos resolver vários problemas de sincronização





Exemplo de uso de Semáforo

- Solução para o problema da seção crítica
  - Criar um semáforo "**mutex**" inicializado em 1

```
wait(mutex);
    SC
signal(mutex);
```

- Considere P_1 e P_2 que executam dois trechos S_1 e S_2, e o requerimento é que S_1 deve acontecer antes de S_2
  - Cria o semáforo "**synch**" inicializado em 0

```
P1:
    S1;
    signal(synch);
P2:
    wait(synch);
    S2;
```





Implementação de Semáforo

- Deve garantir que dois processos não podem executar **`wait()`** e **`signal()`** no mesmo semáforo ao mesmo tempo
- Logo, a implementação se torna a parte crítica do problema onde **`wait`** e **`signal`** são colocados na seção crítica
- Pode haver **busy waiting** na implementação
- Aplicações passam muito tempo na seção crítica
  - **Não é** uma boa solução
  - Alternativa: bloquear o processo em uma fila de espera pelo semáforo





Implementação de Semáforo sem Busy Waiting

- Para cada semáforo, associa uma fila de espera
- Cada dado em uma fila de espera possui dois elementos:
  - Valor (inteiro)
  - Ponteiro para próximo elemento da lista
- Duas operações:
  - **block** – coloca o processo chamando a operação em uma fila de espera
  - **wakeup** – remove um dos processos da fila de espera e o coloca na fila de prontos





Implementação sem Busy Waiting

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```





Problemas com Semáforos

- Uso incorreto:
  - **`signal(mutex)`** …. **`wait(mutex)`**
  - **`wait(mutex)`** … **`wait(mutex)`**
  - Omitir **`wait(mutex)`** e/ou **`signal(mutex)`**
- Esses são exemplos do que pode ocorrer caso semáforos e outras ferramentas não forem utilizadas corretamente



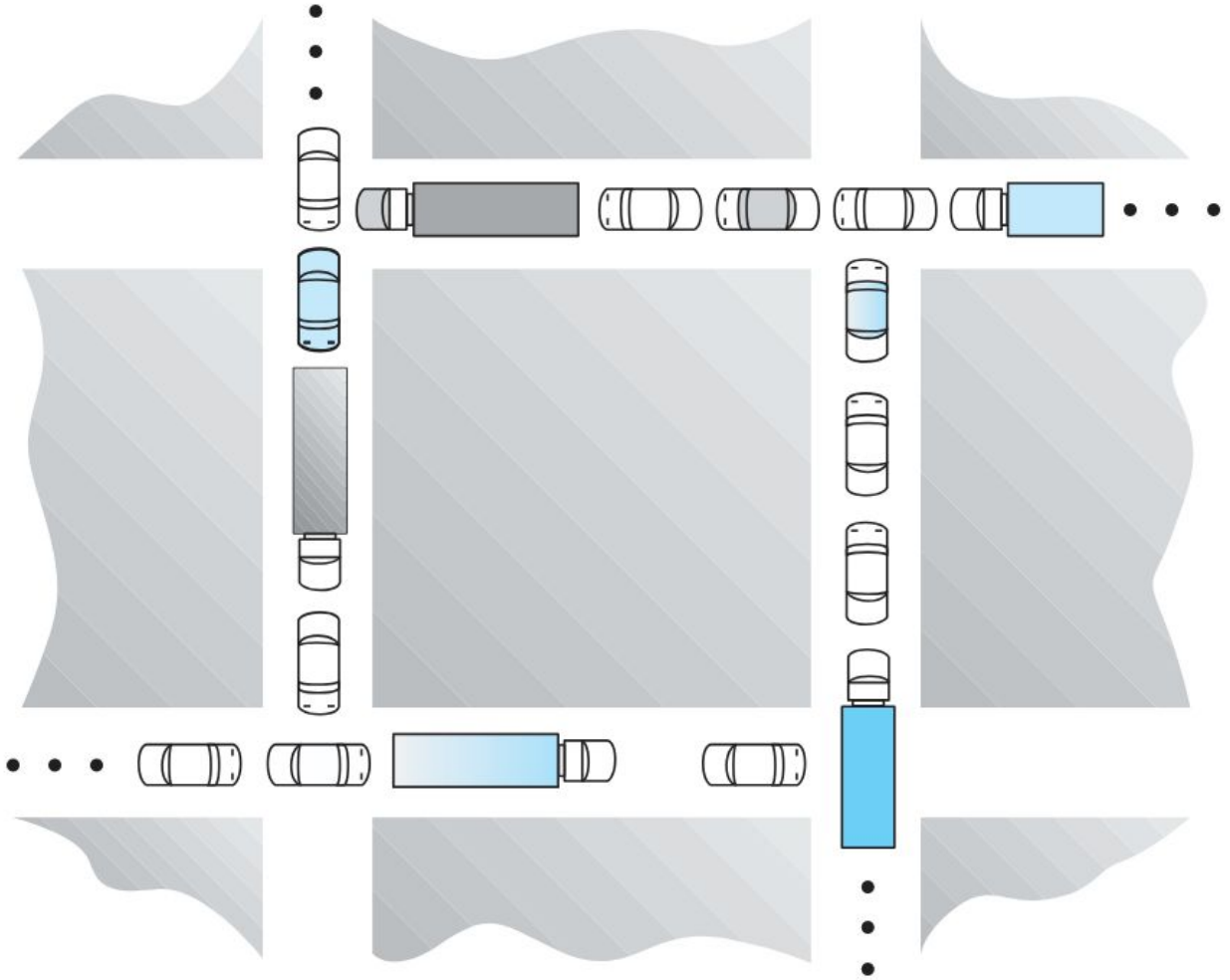


Liveness

- Processos podem esperar indefinidamente esperando uma ferramenta de sincronização como um mutex ou semáforo
- Esperar indefinidamente viola os critérios de progresso e espera-limitada discutidos anteriormente
- **Liveness** se refere ao conjunto de propriedades que o sistema deve satisfazer para garantir que processos tenham progresso
- Espera indefinida é um exemplo de falha de *liveness*



Deadlock





Exemplo de deadlock

- **Deadlock** – dois ou mais processos esperam indefinidamente por um evento que pode ser causado por um dos processos em espera
- Sejam ***S*** e ***Q*** dois semáforos inicializados para 1

P_0	P_1
`wait(S);`	`wait(Q);`
`wait(Q);`	`wait(S);`
...	...
`signal(S);`	`signal(Q);`
`signal(Q);`	`signal(S);`

- Considere que P_0 executa wait(S) e, logo após, P_1 wait(Q). Quando P_0 executa wait(Q), precisa esperar P_1 executar signal(Q)
- Entretanto, P_1 está esperando P_0 executar signal(S).
- Como as operações signal() nunca serão executadas, P_0 e P_1 estão em **deadlock**.

